Standardizing Hybrid Automata: From Theoretical Foundations to Real-World Implementation Frameworks

A Pilot Literature Review on the Development of a Unified Framework for Hybrid Automata Development and Simulation

Ryan McKee

February 2026

Abstract

Hybrid automata are formal models for dynamical systems with both discrete and continuous components [12], widely used across research and industry for applications ranging from control systems and robotics to financial algorithms and cyber-physical systems. A recent example is the modelling of COLREG-compliant autonomy for maritime vehicles [19], where hybrid automata capture both the discrete decision-making of the vessel and its continuous motion dynamics.

This theoretical framework provides a powerful method for modelling complex systems, enabling algorithmic problem-solving without human intervention, while maintaining transparency for human operators to understand the system's decision-making process.

Despite significant advances in artificial intelligence (AI), particularly through large-scale models like OpenAI's GPT series [16] that demonstrate human-like reasoning, challenges remain for real-world deployment. AI systems often suffer from limited transparency, explainability, and reliability, particularly in novel situations, due to the black-box nature of deep neural networks and sensitivity to outliers in training data.

Given these challenges, hybrid automata remain one of the most robust tools for complex system design. However, while the theoretical framework is well-established, there is a lack of standardisation in translating it into practical applications. Different implementations use varying algorithms, making cross-comparison and adoption challenging.

In this pilot literature review we examine the current state of hybrid automata development and simulation, identify gaps in the field, and propose a unified framework for their design and deployment. Such a framework aims to standardise the development process and enhance the efficiency, effectiveness, and applicability of hybrid automata across research and industry.

0.1 Introduction

Hybrid automata are formal models for dynamical systems that combine discrete modes with continuous dynamics [12]. They have been widely adopted in both research and industry for modelling complex hybrid systems such as control systems, autonomous vehicles, and cyber-physical applications. For example, hybrid automata have been used to model decision-making and motion dynamics in maritime autonomy under COLREG rules [14, 19]. This modelling framework provides a transparent, modular way to design and analyse systems where logic transitions and physical processes interact.

In recent years, machine learning (ML) and large AI models (e.g. GPT-4) have demonstrated remarkable capabilities on many tasks, but they often act as "black boxes" with limited interpretability. In contrast, hybrid automata yield inherently explainable models (states and transitions are explicit) and can provide formal guarantees on behaviour. Consequently, hybrid automata remain a powerful tool for safety-critical and complex system design. However, despite a rich theoretical foundation, there is a lack of standardisation in how hybrid automata are implemented and deployed. Different research groups and industries often develop their own modelling frameworks or code libraries, without a common API or data format. In this review, we survey the state-of-the-art in hybrid automaton development and simulation to identify gaps and motivate the need for a unified framework.

Importance and Societal Relevance

The importance of hybrid automata extends well beyond academic interest. Hybrid models underpin safety-critical infrastructure in transport, medicine, and energy. In automotive systems, hybrid automata are used to verify the correctness of anti-lock braking and adaptive cruise control algorithms [1]. In medical devices, they govern the behaviour of pacemakers and insulin pumps, where erroneous mode transitions could have life-threatening consequences. In energy systems, hybrid models describe the switching behaviour of smart-grid controllers. As autonomous systems from self-driving vehicles to unmanned maritime and aerial platforms become increasingly prevalent, the need for formally verified hybrid control logic grows accordingly. The ability to reason about both continuous physical dynamics and discrete control logic within a single, verifiable framework directly impacts public safety and quality of life.

0.2 Background

Formal Definition

A hybrid automaton H can be defined as a tuple $H = (Q, X, f, Init, Inv, E, G, R)$, where Q is the set of discrete modes, X is the continuous state space, f defines the continuous dynamics in each mode, $Init$ specifies initial states, Inv assigns an invariant set to each mode, E is the set of transitions (edges) between modes, G assigns guard conditions to edges, and R assigns reset maps on transitions [12]. Informally, the system resides in some mode $q \in Q$ and evolves according to $\dot{x} = f(q, x)$ as long as the state satisfies the invariant $Inv(q)$. When the continuous state reaches a guard $G(q, q')$, the automaton may take a discrete jump to mode q', resetting the continuous state via $R(q, q', x)$. This hybrid formalism generalises finite automata by adding differential equation dynamics and state resets [2].

Illustrative Example

A simple example is a thermostat system with modes $\{On, Off\}$. The continuous variable is temperature x. In mode On, the heater is active and the temperature evolves as $\dot{x} = 5 - 0.1x$ until x reaches an upper bound (invariant $x \leq 22$). In mode Off, the heater is inactive and $\dot{x} = -0.1x$ until x reaches a lower bound (invariant $x \geq 18$).

Transitions between On and Off are governed by guards with hysteresis: $Off \to On$ occurs when $x < 19$, and $On \to Off$ occurs when $x > 21$. No reset is applied to x on transitions (i.e., R is the identity map).

This example illustrates how hybrid automata capture both the discrete controller (heater on/off) and the continuous physics (temperature dynamics) in a single model, as visualised in Figure 1.

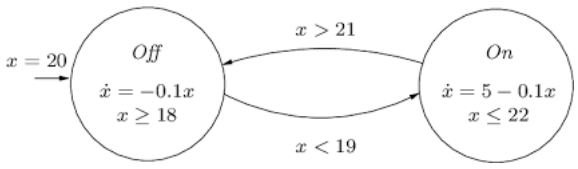

Figure 1: A hybrid automaton model of a thermostat. Figure adapted from ResearchGate [18].

0.3 State of the Art

Historical Development

The theoretical foundations of hybrid automata were established in the early 1990s. Alur et al. [2] introduced the algorithmic framework for reasoning about hybrid systems, and Henzinger's landmark 1996 paper [12] formalised the mathematical definition that remains standard today. This period also saw early decidability results for restricted classes of hybrid automata, such as timed and rectangular automata, which established the boundaries of algorithmic analysis [12].

Through the late 1990s and early 2000s, the focus shifted toward building the first practical verification tools. PHAVer [10], released in 2007, provided the first widely used tool for exact polyhedral verification of linear hybrid automata. CORA [1], introduced in 2010 as part of Althoff's doctoral work at Technische Universität München, provided MATLAB-based reachability analysis for linear systems and became a benchmark tool for automotive safety verification.

The early 2010s saw a significant maturation of the tool landscape. SpaceEx [11], developed at the VERIMAG laboratory in Grenoble (a joint CNRS/Université Grenoble Alpes/Grenoble INP research unit), became the de facto standard for scalable verification of affine hybrid systems following its 2011 CAV publication. Flow* [8], developed by Chen, Ábrahám, and Sankaranarayanan (University of Colorado Boulder), extended reachability analysis to nonlinear dynamics using Taylor models. C2E2 [9], produced by Fan, Qi, and Mitra at the University of Illinois Urbana-Champaign (UIUC), offered simulation-based safety checking for Stateflow models, narrowing the gap between industrial design tools and formal methods.

More recently, the mid-2010s to present has seen a shift toward Python-based frameworks and multi-agent scenarios. The VERSE library [15], developed at UIUC in 2023, provides a scenario-based verification framework in Python, lowering the barrier for newcomers. Ariadne [3], maintained by groups at the University of Verona and CWI Amsterdam, offers Python bindings for rigorous CPS analysis. KeYmaera X [17], developed by Platzer at Carnegie Mellon University (CMU), introduced differential dynamic logic as a proof calculus for hybrid programs, representing a complementary logical approach to the automaton-based tools.

Key Research Groups

Several leading groups define the international landscape of hybrid systems research. The VERIMAG laboratory (Grenoble, France), founded by Joseph Sifakis (Turing Award, 2007), has produced SpaceEx and PHAVer and remains one of the most prolific contributors to hybrid verification. The group of Thomas A. Henzinger, now at IST Austria, continues to produce foundational theoretical results. At UIUC, the group led by Sayan Mitra is responsible for C2E2 and VERSE, with a strong focus on cyber-physical and multi-agent systems. Matthias Althoff's group at TU Munich leads industrial applications of reachability analysis, particularly in the automotive domain. At CMU, André Platzer's group pursues logical approaches to verified hybrid control.

Within the United Kingdom, the group of Wasif Naeem at Queen's University Belfast (QUB) has applied hybrid automata to autonomous maritime systems and COLREG-compliant guidance [19], representing a nationally significant application of this formalism to safety-critical marine autonomy. Broader hybrid and cyber-physical systems research is also active at Imperial College London (control and verification), the University of Edinburgh (formal methods and embedded systems), and the University of Warwick (control engineering).

Tools and Libraries

A variety of specialised tools and frameworks exist for modelling, verifying, and simulating hybrid systems. Formal analysis tools include **SpaceEx** [11] for reachability of piecewise-linear dynamics, **C2E2** [9] for simulation-based safety checking, and **Flow*** [8] for Taylor-model reachability of nonlinear systems. **CORA** [1] provides MATLAB-based reachability for linear systems, and **PHAVer** [10] handles exact verification for rectangular hybrid automata. The **HYST** translator [4] enables model conversion among several tools, but no universally adopted modelling language has emerged.

There are also hybrid modelling libraries and simulators. HyAuLib [7] provides Modelica components for building hybrid automata. The Hybrid Automata Library (HAL) [6] is a Java library facilitating hybrid spatial models in oncology. MATLAB/Simulink users can employ the Hybrid Toolbox [5] by Bemporad et al. for hybrid controller design.

The **Verse** library [15] represents the most significant recent development in the Python ecosystem, providing a multi-agent hybrid verification framework that integrates with reachability backends. Despite this rich landscape, the

IEEE Control Systems Society hybrid systems tool registry [13] lists dozens of tools that remain highly fragmented, each with its own modelling format and programming environment.

0.4 Research Gaps

Despite the wealth of tools, significant gaps remain in translating hybrid automaton theory into practice. First, there is no standard modelling language or API for hybrid automata. Different tools use different input formats (XML for SpaceEx, MATLAB classes for CORA, custom DSLs for KeYmaera X), making it difficult to reuse models across tools. Projects such as **HYST** [4] have attempted to create common interchange formats, but such efforts require broad community adoption. In many cases, converting industrial models (e.g. Simulink/Stateflow) to formal hybrid automata remains cumbersome and error-prone [9, 4].

Second, existing tools focus on analysis rather than ease of integration. SpaceEx [11] is powerful for verification of affine systems, but is not designed as a development library for embedding in larger software stacks. Similarly, model-based design tools such as Simulink are tied to proprietary platforms. Few tools provide both real-time simulation and offline analysis capabilities in a lightweight, unified package.

Third, there is limited support for iterative design and deployment workflows. Researchers frequently implement custom hybrid automata from scratch for each project, often in Python or C++, without reusing existing libraries. This duplication of effort is well-documented in the hybrid systems community [2, 12] and persists today. The lack of a common framework creates steep learning curves and inhibits collaboration across domains.

Fourth, the connection between formal verification and real-time deployment on robotic or embedded platforms remains weak. Middleware such as ROS2 is standard in robotics, yet no hybrid automaton framework natively supports export to ROS2-compatible execution nodes, creating a gap between verification-time models and deployment-time code.

In summary, hybrid systems practitioners face a persistent divide between elegant formal definitions and the practical code required to simulate and deploy automata in real-world applications.

0.5 Conclusions and Future Work

Hybrid automata are theoretically mature and mathematically rigorous, yet their practical ecosystem remains fragmented. Existing tools provide powerful analysis capabilities, but they differ widely in modelling languages, input formats, and integration workflows. This fragmentation creates a gap between formal hybrid automaton theory and practical deployment in real-world systems.

To address this gap, we propose the development of a unified hybrid automaton framework guided by the following principles:

- **Cross-platform, high-level API:** A Python-based library that enables users to define hybrid automata declaratively using structured data representations or a lightweight domain-specific syntax. The API should uniformly support mode definitions, invariants, guards, reset maps, and continuous dynamics while remaining intuitive and extensible.
- **Simulation and real-time execution:** Support for both offline simulation (for analysis and testing) and real-time execution suitable for embedded systems and robotic platforms. Integration with middleware such as ROS2 would enable seamless transition from simulation to deployment.
- **Visualisation and tooling:** Built-in capabilities for visualising automaton graphs, plotting continuous trajectories, and logging state transitions during execution. Such tooling would enhance interpretability and debugging.
- **Performance and modularity:** A lightweight and modular core architecture allowing users to integrate different numerical solvers or reachability backends without altering the primary API. This ensures flexibility while preserving architectural consistency.
- **Compatibility with formal verification backends:** Although primarily focused on simulation and deployment, the framework should support exporting models to formal analysis tools such as SpaceEx [11] or KeYmaera X [17] to enable safety analysis.

A unified framework of this nature would bridge the divide between elegant formal definitions and practical implementation. By standardising how hybrid automata are specified, executed, and analysed, it would promote model reuse, reproducibility, and cross-domain collaboration. Such infrastructure

would support applications ranging from simple control systems to complex autonomous platforms without requiring developers to reconstruct tooling for each new project.

Future work will focus on designing the modelling language and API specification, prototyping the core architecture in Python, and evaluating the framework on benchmark hybrid systems, including those drawn from the maritime autonomy domain [19] to assess scalability, usability, and interoperability with existing verification tools.

Bibliography

[1] Matthias Althoff. *Reachability Analysis and its Application to the Safety Assessment of Autonomous Cars*. PhD thesis, Technische Universität München, 2010.

[2] Rajeev Alur, Costas Courcoubetis, Thomas A. Henzinger, and Pei-Hsin Ho. Hybrid automata: An algorithmic approach to the specification and verification of hybrid systems. In *Hybrid Systems*, volume 736 of *Lecture Notes in Computer Science*, pages 209–229. Springer, 1993.

[3] Ariadne Development Team. Ariadne: A toolbox for hybrid system analysis. `https://www.ariadne-cps.org/about/`, 2023.

[4] Stanley Bak, Sergiy Bogomolov, and Taylor T. Johnson. HYST: A source transformation and translation tool for hybrid automaton models. In *Proceedings of Hybrid Systems: Computation and Control (HSCC)*, pages 128–133. ACM, 2015.

[5] Alberto Bemporad. Hybrid toolbox for MATLAB. `https://cse.lab.imtlucca.it/~bemporad/hybrid/toolbox/`, 2005.

[6] Ricardo Bravo and Alexander R. A. Anderson. Hybrid automata library: A flexible platform for hybrid modeling. *PLoS Computational Biology*, 16(3):e1007635, 2020.

[7] Lena Buffoni and Peter Fritzson. HyAuLib: Modelling hybrid automata in Modelica. In *Proceedings of the 6th International Modelica Conference*, 2008.

[8] Xin Chen, Erika Ábrahám, and Sriram Sankaranarayanan. Flow*: An analyzer for nonlinear hybrid systems. In *Proceedings of the 25th International Conference on Computer Aided Verification (CAV)*, pages 258–263. Springer, 2013.

[9] Chuchu Fan, Bolun Qi, and Sayan Mitra. C2E2: A verification tool for stateflow models. In *Proceedings of the 22nd International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 68–82. Springer, 2016.

[10] Goran Frehse. PHAVer: Algorithmic verification of hybrid systems past HyTech. In *Proceedings of Hybrid Systems: Computation and Control (HSCC)*, pages 258–273. Springer, 2007.

[11] Goran Frehse, Colas Le Guernic, Alexandre Donzé, Scott Cotton, Radu Ray, Olivier Lebeltel, Rodolfo Ripado, Alexandre Girard, Thao Dang, and Oded Maler. SpaceEx: Scalable verification of hybrid systems. In *Proceedings of the 23rd International Conference on Computer Aided Verification (CAV)*, pages 379–395. Springer, 2011.

[12] T.A. Henzinger. The theory of hybrid automata. In *Proceedings 11th Annual IEEE Symposium on Logic in Computer Science*, pages 278–292, 1996.

[13] IEEE Control Systems Society. Hybrid systems tool registry. `https://ieeecss.org/`, 2024.

[14] International Maritime Organization. Convention on the international regulations for preventing collisions at sea (COLREG). `https://www.imo.org/en/about/conventions/pages/colreg.aspx`, 2024.

[15] Yilun Li, Vishnu Murali, Ashish Tiwari, and Sayan Mitra. VERSE: A python library for reasoning about multi-agent hybrid systems. In *Proceedings of the 35th International Conference on Computer Aided Verification (CAV)*, pages 452–465. Springer, 2023.

[16] OpenAI. GPT-4 technical report, 2024.

[17] André Platzer and Yong Kiam Tan. Differential Equation invariance axiomatization. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–12, 2018.

[18] ResearchGate. A hybrid automaton model of a thermostat. `https://www.researchgate.net/figure/A-Hybrid-Automaton-Model-of-a-Thermostat_fig1_4053758`, 2026.

[19] Bhawana Singh, Karim Ahmadi Dastgerdi, Nikolaos Athanasopoulos, Wasif Naeem, and Benoit Lecallard. Safe COLREGs-aware finite-time guidance and control of marine vehicles. *Ocean Engineering*, 351:123919, 2026.